

Artificial Intelligence and Expert Systems

## Task 3 Report

Stanisław Marciniak & Krzysztof Muszyński

---

### Overview

#### *Fifteen puzzle:*

The 15 puzzle is a sliding puzzle, it has 15 square tiles numbered 1 to 15 in a frame that is 4 tile positions high and 4 tile positions wide - leaving one position empty. Tiles can only be moved vertically or horizontally into the empty position (moving tiles diagonally is not allowed).

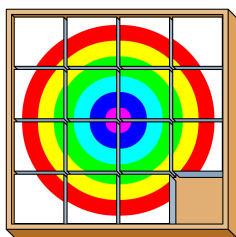
The goal of the puzzle is to achieve a 'solved state' of the puzzle by moving the tiles one by one, ideally in the least amount of moves.

The solved state of the puzzle entails having the tiles positioned in a sequential order from left to right, row by row, with the last position (bottom right of the puzzle) left empty.



There do exist multiple variations of the fifteen puzzle. The puzzle can be adapted by simply changing the amount of rows and columns (such as the 8 puzzle) , or by changing the form slightly (for example substituting numbers for parts of an image

<https://murhafsousli.github.io/8puzzle/#/>)



#### *Task given:*

The given task required proposing a solution to the "Fifteen Puzzle" problem using a number of given algorithms: Breadth-first search, Depth-first search, Iterative deepening DFS, Best-first strategy, A\* strategy, SMA\* strategy.

The solution to the puzzle should be shown as the sequence of moves required to achieve the solved state from the initial state of the puzzle.

---

## Implementation

To implement the solution we used the Python programming language.

The main function works in a linear fashion. The main steps are:

- parse arguments to get the number of rows and columns as well as the desired solving algorithm to be tested from the user
- calling a function to generate a randomly shuffled grid
- printing the shuffled grid for the user to see
- calling the desired solving algorithm function and printing: the solving sequence and time taken to solve.

It is worth mentioning that our program generates random solvable puzzle grids by starting with a solved grid of that dimension, and shuffling the pieces using a random amount of random legal moves. This ensures that each generated puzzle is solvable.

The program has a defined function for each of the solving algorithms. They will be described in the next section of the report.

---

## Algorithms

### ***Breadth-first search***

The breadth first search solution is one of the simpler solutions.

It works by:

- taking in the current state of the grid,
- checking if its in a solved state (in that case returning),
- checking what are the possible legal moves in that grid state,
- then performing all the possible moves on copies of that board
- and repeating the operation for each of the copied grids until one of them reaches a solved state.

The function is optimized by:

- not repeating positions:
  - using a simple check weather the next move would put the piece back where it was one move before,
  - and a more complex scan to see if throughout the process we haven't seen that board position before (using a set of all previously visited grid states)

***Analysis:***

This algorithm is very simple to understand, and works well with smaller dimension puzzles (e.g. 2x2 or 3x3) but struggles with larger ones including the 15 puzzle outlined in the title of the task.

This issue stems from the exponential nature of the algorithm, and according to sources it's simply the wrong algorithm for solving large puzzles of this type.

This algorithm is not very memory efficient but produces shortest solution paths.

This algorithm tries to get to the solution in a brute force manner, going layer by layer through all possible states of the puzzle until reaching a solution.

Using this formula:

$$\text{Reachable states} = (N^2)! / 2$$

where:  $N^2$  is the number of tiles

We can calculate the number of all possible reachable states of the puzzle grid given its dimensions.

Thus, for a **2x2** grid the number will be equal to **12**  
for a **3x3** grid the number will be equal to: **181,440**  
and for a **4x4** it will be much more: **10,461,394,944,000**

As we can see, even a difference of one tile per dimension results in an astounding difference in the number of states the algorithm will need to go through to reach a solution, which renders it unsuitable for larger puzzles.

## ***Depth-first search***

Depth-first search is similar to the breadth-first search algorithm, but instead of going layer by layer (exploring multiple paths simultaneously) it picks one path and sticks to it until it reaches the solution, or if it hits a dead end (previously visited position) it backtracks to the last move and tries to go in that path.

Algorithm:

- check if the grid is solved (if so return)
- mark current position as visited (to know when we hit a dead end and optimize the algorithm)
- check all possible legal moves
- for each move recursively call itself after making the move

*Analysis:*

This algorithm seems to perform even more poorly than the breath-first search algorithm. During testing we were unable to reliably get a solution to even a 3x3 grid puzzle due to the

maximum recursion depth being exceeded.

The largest puzzle size it was able to solve was 2x3.

The average number of moves required to solve the puzzle also turned out higher than in BFS algorithm (in 2x2 grids: 7 moves average for DFS vs 3.8 moves in BFS)

However, for smaller size puzzles such as 2x2 this algorithm took slightly less time than BFS to find the solution (0.00004668 seconds for DFS and 0.000055874 for BFS resulting in a 0.000009194 second difference on average)

When it comes to efficiency, the algorithm is very memory efficient but outputs very long solution paths.

Overall this algorithm seems unusable for puzzles larger than 2x3 grids, although it does seem like for smaller grids such as 2x2 it might be more time efficient than the BFS algorithm.

### ***Iterative deepening DFS***

Iterative deepening DFS is an algorithm that combines the pros of both BFS and DFS algorithms. It is memory efficient like DFS and produces shortest paths like BFS.

It works by performing the DFS algorithm on the puzzle with a set maximum depth, increasing the depth by one each time the solution is not found.

#### ***Analysis:***

This algorithm combines the pros of both BFS and DFS algorithms while mitigating their flaws.

The results indicate it performs better than both the previous ones by themselves.

The results show both short solution paths as well as relatively low time complexity for smaller puzzles.

Although we did manage to get a solution to a 4x4 puzzle we assume that was mostly a coincidence and due to lucky starting configuration, we were not able to get more solutions for such large puzzles in a reasonable amount of time, indicating it's still not the best suited algorithm for the 15 puzzle outlined in the title of the task.

For smaller size puzzles, however, it does seem to be a good choice as it combines finding the shortest path with low memory complexity.

### ***Best-First Search***

Best-First Search (Greedy Best-First) is an informed algorithm that always expands the node which appears to be closest to the goal, according to the heuristic function  $h(n)$ .

It uses the evaluation function:

$$f(n)=h(n)$$

Thus, the algorithm does not take into account the cost of the path so far. It behaves greedily, making decisions based solely on the estimated closeness to the final state.

Algorithm:

- start with the initial puzzle state in a priority queue
- always pick the state that looks closest to the goal according to the heuristic
- check if the current state is solved (if so return the solution path)
- mark the current state as visited
- generate all possible successor states by making legal moves
- for each new state, calculate how close it appears to the goal using the heuristic
- add all new states to the priority queue, keeping them ordered by their heuristic values
- repeat until solution is found

Often fast in practice when heuristic is strong. Not guaranteed to find the optimal solution. May get stuck in misleading heuristic “traps”. Uses a priority queue sorted purely by  $h(n)$ .

Impact of heuristics

$h_0$  (uninformed):

Reduces Best-First to random blind search -> extremely poor performance.

Misplaced tiles:

Provides some guidance but still weak: often produces long paths.

Manhattan distance:

Gives the strongest performance: significantly faster and more directed.

## **A\* Search**

A\* is a heuristic search algorithm that finds optimal solutions if the heuristic is admissible. It uses a combined evaluation function:

$$f(n) = g(n) + h(n)$$

where:

$g(n)$  - cost from the initial state to the current node (path length),

$h(n)$  - heuristic estimate of distance to goal.

Algorithm:

- start with the initial puzzle state, tracking both the actual path cost and estimated distance to goal
- always pick the state with the lowest combined score (actual cost so far plus estimated remaining cost)
- check if the current state is solved (if so return the solution path)

- check if this state was already visited with a shorter path (if so skip it to avoid redundant work)
- mark the current state as visited along with its path cost
- generate all possible successor states by making legal moves
- for each new state, calculate the actual cost to reach it (previous cost plus one move)
- for each new state, also calculate the estimated remaining cost using the heuristic
- add all new states to the priority queue, ordered by their combined score
- repeat until solution is found

A\* balances:

exploration of short paths ( $g(n)$ ),

exploitation of states that look promising ( $h(n)$ ).

If  $h(n)$  is admissible and consistent, A\* is optimal and complete. Expands fewer nodes than uniform-cost search (Dijkstra). High memory consumption must store all generated nodes.

Impact of heuristics

$h_0$ :

A\* becomes Dijkstra's algorithm. Guarantees optimality but is extremely slow.

Misplaced tiles:

Weak admissible heuristic  $\rightarrow$  many expansions  $\rightarrow$  high memory usage.

Manhattan distance:

Strong admissible heuristic. Reduces search dramatically.

### ***SMA\* (Simplified Memory-Bound A\*)***

SMA\* is a memory-bounded version of A\*. It aims to preserve A\*'s optimality while using limited memory.

It maintains a fixed-size frontier and behaves as follows:

If memory is available, SMA\* expands nodes like A\*.

If memory is full, it deletes the worst node (highest f-value).

It backs up the f-value into its parent so the path can be reconsidered later if needed.

When the algorithm needs to regenerate the deleted node, it re-expands it on demand.

Algorithm:

- start with the initial puzzle state, tracking both actual path cost and estimated distance to goal
- set a maximum memory limit for the number of states that can be stored
- always pick the state with the lowest combined score (actual cost plus estimated remaining cost)
- before expanding, check if memory limit is exceeded

- if memory is full, remove the states with highest combined scores (worst prospects) to free up space
- check if the current state is solved (if so return the solution path)
- check if this state was already visited with a shorter path (if so skip it)
- mark the current state as visited along with its path cost
- generate all possible successor states by making legal moves
- for each new state, calculate both actual cost and estimated remaining cost
- add all new states to the priority queue, ordered by their combined score
- repeat until solution is found

SMA\* guarantees optimality as long as sufficient memory is available.

When memory is too small, SMA\* will still return a solution but not necessarily an optimal one.

Memory-bounded (unlike A\*). Can handle much deeper puzzles than A\* with restricted resources. May need to regenerate forgotten nodes multiple times. When memory is too small, it behaves similarly to Best-First Search.

Impact of heuristics

The influence of heuristics is identical to A\*, but memory limits may prevent SMA\* from fully exploiting them.

## Test

We tested 12 algorithm variants on 30 randomly generated puzzles (10× 2x2, 10× 3x3, 10× 4x4) with 5-second timeout per puzzle.

Best Overall Performance:

- Best-First (h=2): 100% success rate, 47 moves avg, 11ms avg - fastest reliable solver
- Best-First (h=1): 100% success rate, 65 moves avg, 34ms avg

Poor Performers:

- DFS: Only 33% success despite being fastest (0.02ms) - gets lost in deep paths
- Uninformed methods (h=0): Struggle with larger puzzles due to exponential search space

Key Insight: Manhattan distance heuristic (h=2) dramatically improves performance.

Best-First sacrifices solution optimality for speed/reliability, while A\* maintains near-optimal solutions with good heuristics.

```
=====
OVERALL PERFORMANCE SUMMARY
=====
```

| Algorithm        | Total Solved | Avg Moves | Avg Time (ms) |
|------------------|--------------|-----------|---------------|
| BFS              | 20/30        | 11.4      | 248.8727      |
| DFS              | 10/30        | 6.1       | 0.0230        |
| IDFS             | 18/30        | 9.9       | 636.6477      |
| Best-First (h=0) | 20/30        | 11.4      | 275.5518      |
| Best-First (h=1) | 30/30        | 64.5      | 34.4907       |
| Best-First (h=2) | 30/30        | 46.8      | 11.1644       |
| A* (h=0)         | 20/30        | 11.4      | 376.9906      |
| A* (h=1)         | 22/30        | 12.7      | 198.4217      |
| A* (h=2)         | 28/30        | 17.1      | 219.0254      |
| SMA* (h=0)       | 20/30        | 11.4      | 289.6004      |
| SMA* (h=1)       | 20/30        | 11.4      | 40.7227       |
| SMA* (h=2)       | 26/30        | 15.7      | 40.5003       |

```
=====
```

```
=====
OVERALL PERFORMANCE SUMMARY
=====
```

| Algorithm        | Total Solved | Avg Moves | Avg Time (ms) |
|------------------|--------------|-----------|---------------|
| BFS              | 19/30        | 12.8      | 497.0484      |
| DFS              | 11/30        | 142.0     | 1.4145        |
| IDFS             | 13/30        | 7.5       | 178.2256      |
| Best-First (h=0) | 19/30        | 12.8      | 522.0859      |
| Best-First (h=1) | 29/30        | 70.5      | 83.3001       |
| Best-First (h=2) | 29/30        | 46.2      | 13.1259       |
| A* (h=0)         | 19/30        | 12.8      | 508.3632      |
| A* (h=1)         | 19/30        | 12.8      | 99.5066       |
| A* (h=2)         | 29/30        | 19.7      | 243.1653      |
| SMA* (h=0)       | 19/30        | 12.8      | 522.6038      |
| SMA* (h=1)       | 19/30        | 12.8      | 112.9017      |
| SMA* (h=2)       | 28/30        | 19.1      | 99.3635       |

```
=====
```

As we can see, all metrics (amount of solved puzzles, average moves length, average time taken to solve) vary between the two tests we performed. It is most likely determined by the fact that we generate the puzzles randomly and some may require more moves to solve than the others, thus making them more complex and causing the variability in our results.

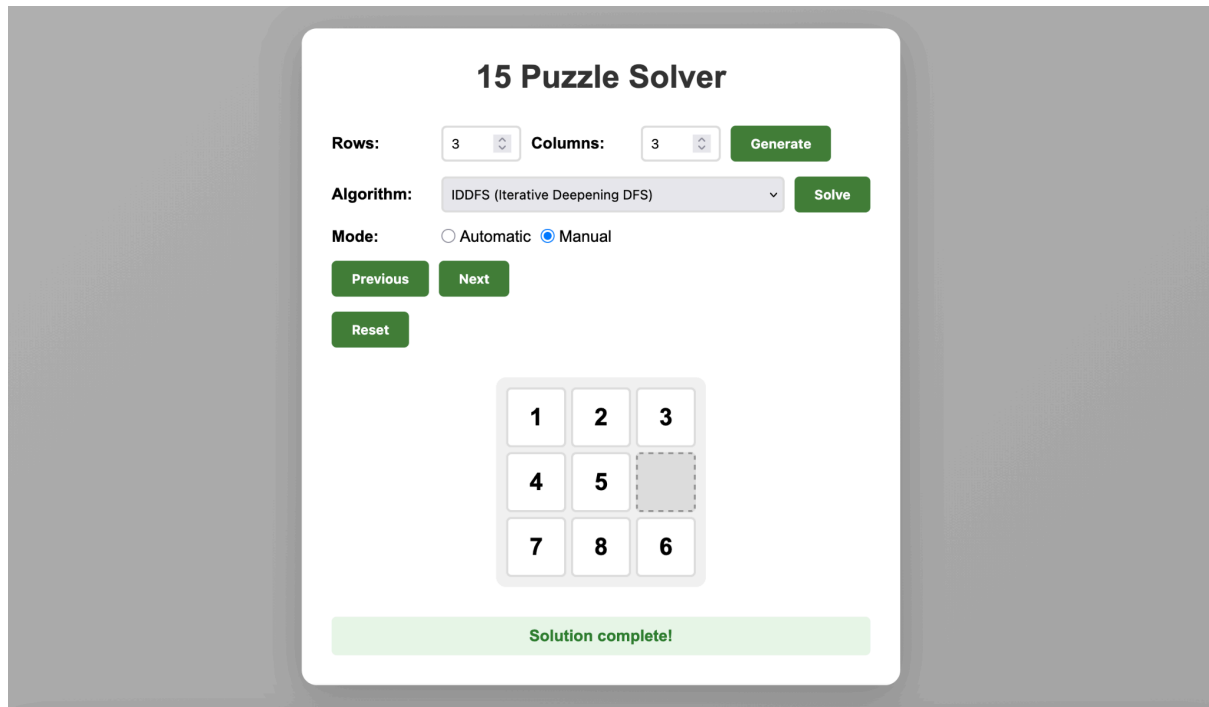
---

### Web application

The task also entailed creating an interactive user interface for the user to be able to see the solution visually by sliding the numbered tiles.  
For this app we created a simple flask backend that uses functions imported from our python program.

The resulting app allows the user to randomly generate a shuffled puzzle, choose an algorithm they want to solve the puzzle and see the animation of the board being solved one piece at a time, or click through all the moves themselves until a solution is reached.

Screenshot:



---

## Sources

[https://en.wikipedia.org/wiki/15\\_puzzle](https://en.wikipedia.org/wiki/15_puzzle)  
<https://ftims.edu.p.lodz.pl/mod/page/view.php?id=11595>  
<https://murhafsousli.github.io/8puzzle/#/>  
<https://www.youtube.com/watch?v=2OHgYpzXKA8>